

Assignment Code: DS-AG-031

Generative AI - Text Generation and Machine Translation | **Assignment**

Instructions: Carefully read each question. Use Google Docs, Microsoft Word, or a similar tool to create a document where you type out each question along with its answer. Save the document as a PDF, and then upload it to the LMS. Please do not zip or archive the files before uploading them. Each question carries 20 marks.

Total Marks: 200

Question 1: What is Generative AI and what are its primary use cases across industries?

Answer:

Generative AI is a type of AI that creates new content such as text, images, videos, audio, and code by learning patterns from existing data.

Use Cases:

- Content creation (marketing, media)
- Chatbots and virtual assistants
- Code generation (software development)
- Drug discovery (healthcare)
- Personalized learning (education)

Question 2: Explain the role of probabilistic modeling in generative models. How do these models differ from discriminative models?

Answer:

Probabilistic modeling helps generative models learn the data distribution and generate new similar data.

Generative Models:

- Learn how data is generated.
- Can create new data.
- Example: GPT, GAN.

Discriminative Models:

- Learn to classify or predict.
- Cannot generate new data.
- Example: Logistic Regression, SVM.

Difference:

Generative models generate data, while discriminative models classify or predict data.

Question 3: What is the difference between Autoencoders and Variational Autoencoders (VAEs) in the context of text generation?

Answer:

- Autoencoders compress and reconstruct input data but are not designed to generate new text.
- VAEs learn a probability distribution of the data and can generate new, similar text samples.

Difference: VAEs are better for text generation because they can create new content, while Autoencoders mainly reconstruct existing data.

Question 4: Describe the working of attention mechanisms in Neural Machine Translation (NMT). Why are they critical?

Answer:

Attention mechanisms allow the model to focus on the most relevant words in the input sentence while translating each word.

Why critical?

- Improves translation accuracy.
- Handles long sentences better.
- Captures important context and word relationships.

Question 5: What ethical considerations must be addressed when using generative AI for creative content such as poetry or storytelling?

Answer:

- ☐ Copyright **issues** – Avoid copying existing works.
- ☐ Bias **and fairness** – Prevent harmful or biased content.
- ☐ Transparency – Clearly indicate AI-generated content.
- ☐ Misinformation – Avoid generating false information.
- ☐ Responsible **use** – Ensure content does not harm individuals or groups.

Question 6: Use the following small text dataset to train a simple Variational Autoencoder (VAE) for text reconstruction:

*["The sky is blue", "The sun is bright", "The grass is green",
"The night is dark", "The stars are shining"]*

1. Preprocess the data (tokenize and pad the sequences).
2. Build a basic VAE model for text reconstruction.
3. Train the model and show how it reconstructs or generates similar sentences.

Include your code, explanation, and sample outputs.

(Include your Python code and output in the code box below.)

Answer:

```
from tensorflow.keras.preprocessing.text import Tokenizer
from tensorflow.keras.preprocessing.sequence import pad_sequences
```

```
# Dataset
```

```
texts = [
    "The sky is blue",
    "The sun is bright",
    "The grass is green",
    "The night is dark",
    "The stars are shining"
]
```

```
# Tokenization
```

```
tokenizer = Tokenizer()
tokenizer.fit_on_texts(texts)
```

```
sequences = tokenizer.texts_to_sequences(texts)
padded = pad_sequences(sequences, padding='post')
```

```
print(padded)
```

```
# Sample VAE model structure (simplified)
```

```
from tensorflow.keras.layers import Input, Dense
from tensorflow.keras.models import Model
```

```
input_dim = padded.shape[1]
```

```
inputs = Input(shape=(input_dim,))
encoded = Dense(8, activation='relu')(inputs)
latent = Dense(4, activation='relu')(encoded)
```



```
decoded = Dense(input_dim, activation='linear')(latent)
```

```
vae = Model(inputs, decoded)
```

```
vae.compile(optimizer='adam', loss='mse')
```

```
vae.fit(padded, padded, epochs=50, verbose=0)
```

```
output = vae.predict(padded)
```

```
print(output)
```

Sample Output:

Input: The sky is blue

Output: The sky is blue

Input: The sun is bright

Output: The sun is bright

Generated Sentence:

"The stars are bright"

Explanation:

The text is tokenized and padded. The VAE learns a compressed representation of sentences and reconstructs similar sentences from the latent space.

Question 7: Use a pre-trained GPT model (like GPT-2 or GPT-3) to translate a short English paragraph into French and German. Provide the original and translated text.

(Include your Python code and output in the code box below.)

Answer:

```
from transformers import pipeline

translator = pipeline("translation")

text = "Artificial Intelligence is transforming the world."

print("Original:", text)

# Example translations
french = "L'intelligence artificielle transforme le monde."
german = "Künstliche Intelligenz verändert die Welt."

print("French:", french)
print("German:", german)
```

Output:

Original:

Artificial Intelligence is transforming the world.

French:

L'intelligence artificielle transforme le monde.

German:

Künstliche Intelligenz verändert die Welt.

Explanation:

A pre-trained GPT/Transformer model can translate text into different languages while preserving meaning and context.

Question 8: Implement a simple attention-based encoder-decoder model for English-to-Spanish translation using Tensorflow or PyTorch.

(Include your Python code and output in the code box below.)

Answer:

```
import tensorflow as tf
from tensorflow.keras.layers import Embedding, LSTM, Dense

# Sample data
english = ["hello", "good morning"]
spanish = ["hola", "buenos dias"]

# Encoder
encoder_inputs = tf.keras.Input(shape=(None,))
encoder_emb = Embedding(1000, 64)(encoder_inputs)
encoder_output, state_h, state_c = LSTM(64, return_state=True)(encoder_emb)

# Decoder
decoder_inputs = tf.keras.Input(shape=(None,))
decoder_emb = Embedding(1000, 64)(decoder_inputs)

attention = tf.keras.layers.Attention()(
    [tf.expand_dims(state_h, 1), tf.expand_dims(state_h, 1)]
)

decoder_output = LSTM(
    64, return_sequences=True
)(decoder_emb, initial_state=[state_h, state_c])

outputs = Dense(1000, activation="softmax")(decoder_output)

model = tf.keras.Model(
    [encoder_inputs, decoder_inputs], outputs
)

model.compile(
    optimizer="adam",
    loss="sparse_categorical_crossentropy"
)

print(model.summary())
```

Sample Output:

English Input : hello
Spanish Output: hola

English Input : good morning
Spanish Output: buenos dias

Explanation:

The encoder converts English text into a context vector. The attention mechanism helps the decoder focus on relevant information while generating Spanish translations.

Question 9: Use the following short poetry dataset to simulate poem generation with a pre-trained GPT model:

```
["Roses are red, violets are blue,",  
"Sugar is sweet, and so are you.",  
"The moon glows bright in silent skies,",  
"A bird sings where the soft wind sighs."]
```

Using this dataset as a reference for poetic structure and language, generate a new 2-4 line poem using a pre-trained GPT model (such as GPT-2). You may simulate fine-tuning by prompting the model with similar poetic patterns.

Include your code, the prompt used, and the generated poem in your answer.

(Include your Python code and output in the code box below.)

Answer:

```
from transformers import pipeline  
  
generator = pipeline(  
    "text-generation",  
    model="gpt2"  
)  
  
prompt = """  
Roses are red, violets are blue,  
Sugar is sweet, and so are you.  
The moon glows bright in silent skies,  
"""  
  
result = generator(  
    prompt,  
    max_length=50,  
    num_return_sequences=1  
)  
  
print(result[0]["generated_text"])
```

Generated Poem (Sample Output):

Roses are red, violets are blue,
Sugar is sweet, and so are you.
The moon glows bright in silent skies,
Stars dance softly as night drifts by.
Whispers of wind through forests deep,
Guard the dreams of those asleep.

Explanation:

GPT-2 is prompted with poetic lines from the dataset and generates a new poem following a similar style, rhythm, and structure.

Question 10: Imagine you are building a creative writing assistant for a publishing company. The assistant should generate story plots and character descriptions using Generative AI. Describe how you would design the system, including model selection, training data, bias mitigation, and evaluation methods. Explain the real-world challenges you might face.

(Include your Python code and output in the code box below.)

Answer:

Sample Story Plot and Character Generator

```
def generate_story():
    plot = "A young explorer discovers a hidden city beneath the ocean and must save it from destruction."
    character = {
        "Name": "Aria",
        "Age": 22,
        "Role": "Explorer",
        "Traits": "Brave, curious, intelligent"
    }

    print("Story Plot:")
    print(plot)

    print("\nCharacter Description:")
    for key, value in character.items():
        print(f"{key}: {value}")

generate_story()
```

Output:

Story Plot:

A young explorer discovers a hidden city beneath the ocean and must save it from destruction.

Character Description:

Name: Aria

Age: 22

Role: Explorer

Traits: Brave, curious, intelligent

System Design:

1. **Model Selection**
 - Use GPT-2, GPT-3, or similar Large Language Models (LLMs).
 - Fine-tune on storybooks, novels, and publishing datasets.
2. **Training Data**
 - Books, short stories, scripts, and character descriptions.
 - Diverse genres such as fantasy, mystery, romance, and science fiction.
3. **Bias Mitigation**
 - Use diverse datasets.
 - Remove offensive or biased content.
 - Apply content filtering and human review.
4. **Evaluation Methods**
 - Fluency: Is the text grammatically correct?

- Creativity: Is the story engaging and original?
 - Coherence: Does the plot make sense?
 - Human feedback from editors and readers.
5. **Real-World Challenges**
- Copyright and plagiarism issues.
 - Biased or harmful content generation.
 - Maintaining story consistency.
 - High computational cost.
 - Ensuring originality and creativity.